

Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Отчёт
По лабораторной работе N1
по дисциплине Администрирование систем и сетей
Вариант: -
Желаемая оценка: 4

Работу выполнили:
Молчанов Федор Денисович Р3413
Пышкин Никита Сергеевич Р3413

Работу принял:
Максимов Андрей Николаевич

Оглавление

Общие сведения	2
Цель работы	2
Среда выполнения и реализация стенда	2
Воспроизводимые конечные состояния	2
Топология и адресация стенда	3
Часть 1. Адресация и маршрутизация IPv4	4
Настройка IPv4-адресации	4
Физические интерфейсы	4
Loopback-интерфейсы	5
Настройка статической маршрутизации	5
Основные маршруты между loopback-интерфейсами	5
Резервный маршрут и отказ линии	7
Маршрут по умолчанию	8
Итоговая конфигурация первой части	8
Адреса интерфейсов	8
Контрольные вопросы первой части	9
Когда статический маршрут добавляется в таблицу IP-маршрутизации?	9
Какой исходный IP будет выбран для ping без явного указания source?	9
Вывод по первой части	9
Часть 2. Маршрутизация OSPF	10
Цель и план второй части	10
Подготовка FRRouting	10
Исходное состояние перед OSPF	10
Настройка базового OSPF	10
Аутентификация OSPF	12
Анонсирование маршрута по умолчанию	12
Управление выбором маршрута с помощью OSPF cost	13
Асимметричный обратный маршрут	14
Проверка отказоустойчивости OSPF	14
Ответ на контрольный вопрос второй части	15
Вывод по второй части	15
Справочная информация по командам и выводу	16
Параметры виртуального стенда	16
Как Linux создаёт маршруты при назначении адреса	16
Параметры команд Linux	16
FRRouting, OSPF и его диагностические поля	17

Общие сведения

Цель работы

Цель лабораторной работы — получить практические навыки настройки IPv4-адресов, loopback-интерфейсов, статической и динамической маршрутизации. В первой части исследуются статические и резервные маршруты, необходимость обратного маршрута и маршрут по умолчанию. Во второй части на том же стенде настраивается OSPF: формирование соседств, распространение маршрутов, MD5-аутентификация, анонсирование маршрута по умолчанию, управление выбором пути с помощью стоимости и автоматическая перестройка маршрутов при отказе линии.

Исходные лабораторные работы рассчитаны на маршрутизаторы Huawei. В данной реализации устройства воспроизведены тремя Docker-контейнерами с Linux. В первой части использовались стандартные средства ядра Linux (ip addr, ip link, ip route, ping, traceroute), а во второй части для реализации OSPF применялся FRRouting 8.4.4 (zebra, ospfd, vtysh).

Среда выполнения и реализация стенда

Хостовая система: Arch Linux. Для каждого маршрутизатора создан отдельный Docker-контейнер на базе Ubuntu 24.04. В образ установлены iproute2, iputils-ping, traceroute, procs, tcpdump, а для второй части — frr и tini. Контейнеры запускались без стандартной Docker-сети (--network none), с включенной пересылкой IPv4-пакетов (net.ipv4.ip_forward=1). Для работы FRRouting контейнерам были предоставлены capabilities NET_ADMIN, NET_RAW и SYS_ADMIN.

Виртуальные линии между маршрутизаторами созданы как пары veth. Каждый конец пары перенесен в network namespace соответствующего контейнера и переименован в eth1 или eth2.

```
# пример создания одного виртуального линка R1-R2
ip link add r1-r2 type veth peer name r2-r1
ip link set r1-r2 netns <PID_R1>
ip link set r2-r1 netns <PID_R2>
```

Такая реализация позволяет работать с независимыми интерфейсами и таблицами маршрутизации каждого устройства, при этом все контейнеры используют одно ядро Linux хоста.

Воспроизводимые конечные состояния

Для конечных состояний двух частей подготовлены разные образы и entrypt-скрипты:

Часть	Образ и конфигурация	Скрипт запуска
1	Dockerfile.part1, router-part1-entrypt.sh	launch-part1.sh создаёт na-part1-r1, na-part1-r2, na-part1-r3
2	Dockerfile.part2, router-part2-entrypt.sh	launch-part2.sh создаёт na-part2-r1, na-part2-r2, na-part2-r3

Docker-образ не может сам хранить veth-пары и таблицы конкретного network namespace: эти объекты существуют только во время работы контейнеров. Поэтому entrypt назначает адреса и маршруты, а launch-скрипт создаёт три контейнера, переносит шесть концов veth в нужные namespaces и ожидает готовности протоколов.

```
# Воспроизвести конец первой части со статическими маршрутами
root@host:/lab1# ./launch-part1.sh

# Проверить таблицу и связность
root@host:/lab1# docker exec na-part1-r1 ip route show
root@host:/lab1# docker exec na-part1-r1 \
    ping -c 3 -I 10.0.1.1 10.0.1.2

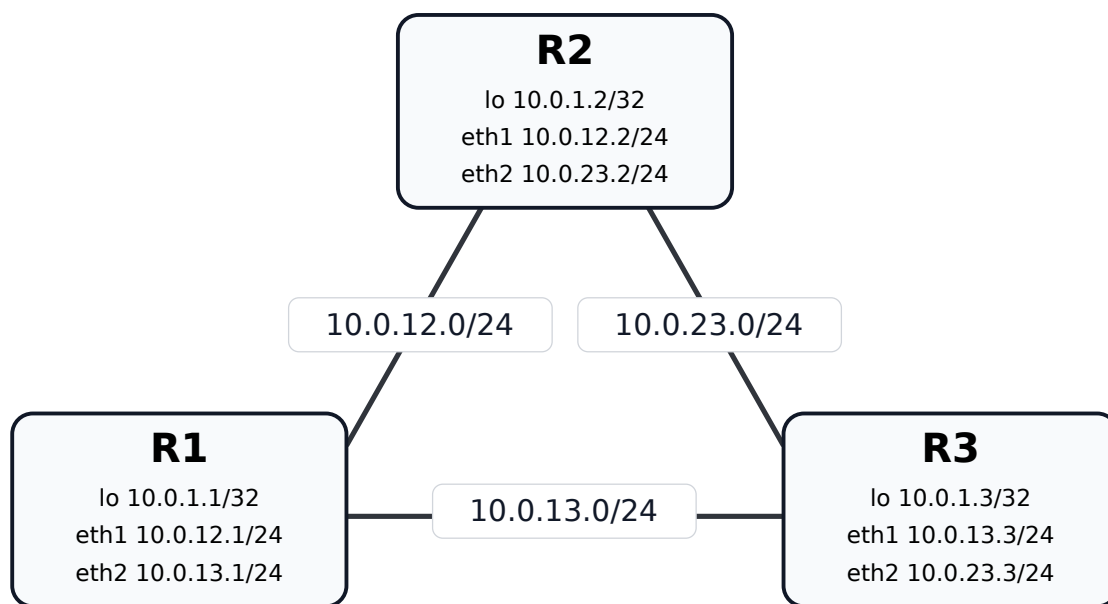
# Воспроизвести конец второй части с OSPF
root@host:/lab1# ./launch-part2.sh

# Проверить соседства и выбранный путь
```

```
root@host:/lab1# docker exec na-part2-r1 \
  vtysh -c "show ip ospf neighbor"
root@host:/lab1# docker exec na-part2-r1 \
  vtysh -c "show ip route 10.0.1.2/32"
```

Имена контейнеров и veth-пар различаются, поэтому оба конечных стенда могут работать одновременно. На проверенном стенде первой части R1 достигал loopback R2 через default route; на стенде второй части оба соседа R1 находились в Full, а маршрут 10.0.1.2/32 с cost 20 проходил через R3 (10.0.13.3).

Топология и адресация стенда



Связи реализованы парами veth; R1, R2 и R3 — отдельные Docker network namespace.

Figure 1: Реализованная топология и адресация

Использованы три транзитные сети: 10.0.12.0/24, 10.0.13.0/24, 10.0.23.0/24. Для имитации конечных сетей/клиентов на loopback-интерфейсах используются адреса 10.0.1.1/32, 10.0.1.2/32, 10.0.1.3/32.

Часть 1. Адресация и маршрутизация IPv4

Настройка IPv4-адресации

Физические интерфейсы

Настройка адресов выполнялась следующими командами:

```
# R1
ip addr add 10.0.12.1/24 dev eth1
ip addr add 10.0.13.1/24 dev eth2

# R2
ip addr add 10.0.12.2/24 dev eth1
ip addr add 10.0.23.2/24 dev eth2

# R3
ip addr add 10.0.13.3/24 dev eth1
ip addr add 10.0.23.3/24 dev eth2
```

Сразу после назначения адресов была выведена краткая сводка интерфейсов и обе таблицы, в которые ядро добавляет связанные с адресом маршруты:

```
root@R1:~# ip -br addr
lo                UNKNOWN          127.0.0.1/8 ::1/128
eth1@if9          UP                10.0.12.1/24 fe80::bc57:9aff:fe0d:7eb8/64
eth2@if11         UP                10.0.13.1/24 fe80::8888:9fff:fe06:af62/64

root@R1:~# ip route show
10.0.12.0/24 dev eth1 proto kernel scope link src 10.0.12.1
10.0.13.0/24 dev eth2 proto kernel scope link src 10.0.13.1

root@R1:~# ip route show table local
local 10.0.12.1 dev eth1 proto kernel scope host src 10.0.12.1
broadcast 10.0.12.255 dev eth1 proto kernel scope link src 10.0.12.1
local 10.0.13.1 dev eth2 proto kernel scope host src 10.0.13.1
broadcast 10.0.13.255 dev eth2 proto kernel scope link src 10.0.13.1
```

Аналогичные Huawei «три прямых маршрута» в Linux распределены между таблицами: префикс подключенной сети находится в main, а локальный /32 и broadcast-адрес — в local.

Связность непосредственно подключенных сегментов проверялась до настройки маршрутов к loopback. Флаг -с 3 ограничивает проверку тремя запросами; без него Linux ping работал бы до прерывания сочетанием Ctrl+C.

```
root@R1:~# ping -c 3 10.0.12.2
PING 10.0.12.2 (10.0.12.2) 56(84) bytes of data.
64 bytes from 10.0.12.2: icmp_seq=1 ttl=64 time=0.052 ms
64 bytes from 10.0.12.2: icmp_seq=2 ttl=64 time=0.040 ms
64 bytes from 10.0.12.2: icmp_seq=3 ttl=64 time=0.098 ms

--- 10.0.12.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2085ms
rtt min/avg/max/mdev = 0.040/0.063/0.098/0.025 ms

root@R1:~# ping -c 3 10.0.13.3
PING 10.0.13.3 (10.0.13.3) 56(84) bytes of data.
64 bytes from 10.0.13.3: icmp_seq=1 ttl=64 time=0.058 ms
64 bytes from 10.0.13.3: icmp_seq=2 ttl=64 time=0.037 ms
64 bytes from 10.0.13.3: icmp_seq=3 ttl=64 time=0.035 ms

--- 10.0.13.3 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2041ms
rtt min/avg/max/mdev = 0.035/0.043/0.058/0.010 ms
```

Полностью приведена одна однотипная проверка; для второго физического сегмента оставлена итоговая статистика. На этом этапе она подтверждает только исправность veth-линий и адресацию соседей.

Loopback-интерфейсы

В Linux использован уже существующий интерфейс `lo`, которому назначены дополнительные адреса:

```
# R1
ip addr add 10.0.1.1/32 dev lo
# R2
ip addr add 10.0.1.2/32 dev lo
# R3
ip addr add 10.0.1.3/32 dev lo
```

Собственный loopback-адрес помещается Linux в локальную таблицу маршрутизации. После его создания удалённый loopback всё ещё недоступен:

```
root@R1:/# ip -br addr show lo
lo                UNKNOWN          127.0.0.1/8 10.0.1.1/32 ::1/128

root@R1:/# ip route show table local 10.0.1.1/32
local 10.0.1.1 dev lo proto kernel scope host src 10.0.1.1

root@R1:/# ip route get 10.0.1.2 from 10.0.1.1
RTNETLINK answers: Network is unreachable

root@R1:/# ping -c 3 -I 10.0.1.1 10.0.1.2
PING 10.0.1.2 (10.0.1.2) from 10.0.1.1 : 56(84) bytes of data.

--- 10.0.1.2 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2041ms
```

Флаг `-I 10.0.1.1` выбирает loopback как исходный адрес ICMP-пакетов и является аналогом Huawei `ping -a`. Он нужен не для оформления: без него Linux выбрал бы адрес выходного физического интерфейса, и опыт проверял бы другой обратный маршрут.

Настройка статической маршрутизации

Основные маршруты между loopback-интерфейсами

Полный план статических маршрутов приведён ниже, однако команды выполнялись поэтапно, чтобы наблюдать изменение таблиц:

```
# R1
ip route add 10.0.1.2/32 via 10.0.12.2
ip route add 10.0.1.3/32 via 10.0.13.3

# R2
ip route add 10.0.1.1/32 via 10.0.12.1
ip route add 10.0.1.3/32 via 10.0.23.3

# R3
ip route add 10.0.1.1/32 via 10.0.13.1
ip route add 10.0.1.2/32 via 10.0.23.2
```

Проверка R1–R2 проводилась поэтапно. Сначала была выполнена только первая команда на R1:

```
root@R1:/# ip route add 10.0.1.2/32 via 10.0.12.2

root@R1:/# ip route show
10.0.1.2 via 10.0.12.2 dev eth1
10.0.12.0/24 dev eth1 proto kernel scope link src 10.0.12.1
10.0.13.0/24 dev eth2 proto kernel scope link src 10.0.13.1
```

```

root@R1:/# ip route get 10.0.1.2 from 10.0.1.1
10.0.1.2 from 10.0.1.1 via 10.0.12.2 dev eth1 uid 0
      cache

root@R1:/# ping -c 3 -I 10.0.1.1 10.0.1.2
PING 10.0.1.2 (10.0.1.2) from 10.0.1.1 : 56(84) bytes of data.

--- 10.0.1.2 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2039ms

root@R2:/# ip route get 10.0.1.1 from 10.0.1.2
RTNETLINK answers: Network is unreachable

```

ip route get моделирует выбор маршрута для одного пакета, не отправляя его. Параметр from задаёт адрес источника для этого выбора. R1 уже выбирает 10.0.12.2, но R2 не имеет маршрута к источнику 10.0.1.1, поэтому ICMP echo reply вернуть невозможно. Затем отдельно был добавлен обратный маршрут и сразу выведены обе таблицы:

```

root@R2:/# ip route add 10.0.1.1/32 via 10.0.12.1

root@R1:/# ip route show
10.0.1.2 via 10.0.12.2 dev eth1
10.0.12.0/24 dev eth1 proto kernel scope link src 10.0.12.1
10.0.13.0/24 dev eth2 proto kernel scope link src 10.0.13.1

root@R2:/# ip route show
10.0.1.1 via 10.0.12.1 dev eth1
10.0.12.0/24 dev eth1 proto kernel scope link src 10.0.12.2
10.0.23.0/24 dev eth2 proto kernel scope link src 10.0.23.2

root@R1:/# ping -c 3 -I 10.0.1.1 10.0.1.2
PING 10.0.1.2 (10.0.1.2) from 10.0.1.1 : 56(84) bytes of data.
64 bytes from 10.0.1.2: icmp_seq=1 ttl=64 time=0.072 ms
64 bytes from 10.0.1.2: icmp_seq=2 ttl=64 time=0.037 ms
64 bytes from 10.0.1.2: icmp_seq=3 ttl=64 time=0.066 ms

--- 10.0.1.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2033ms
rtt min/avg/max/mdev = 0.037/0.058/0.072/0.015 ms

```

Успешный двусторонний обмен требует пути к назначению и пути обратно к источнику. После добавления оставшихся маршрутов таблицы стали следующими:

```

root@R1:/# ip route show
10.0.1.2 via 10.0.12.2 dev eth1
10.0.1.3 via 10.0.13.3 dev eth2
10.0.12.0/24 dev eth1 proto kernel scope link src 10.0.12.1
10.0.13.0/24 dev eth2 proto kernel scope link src 10.0.13.1

root@R2:/# ip route show
10.0.1.1 via 10.0.12.1 dev eth1
10.0.1.3 via 10.0.23.3 dev eth2
10.0.12.0/24 dev eth1 proto kernel scope link src 10.0.12.2
10.0.23.0/24 dev eth2 proto kernel scope link src 10.0.23.2

root@R3:/# ip route show
10.0.1.1 via 10.0.13.1 dev eth1
10.0.1.2 via 10.0.23.2 dev eth2
10.0.13.0/24 dev eth1 proto kernel scope link src 10.0.13.3
10.0.23.0/24 dev eth2 proto kernel scope link src 10.0.23.3

```

Контрольные ping R1->R3 и R2->R3 с соответствующими loopback-источниками дали по три ответа и 0 % потерь.

Проверка	Результат	Назначение
R1 10.0.1.1 -> R2 10.0.1.2	0 % потерь	проверка маршрутов R1R2
R1 10.0.1.1 -> R3 10.0.1.3	0 % потерь	проверка маршрутов R1R3
R2 10.0.1.2 -> R3 10.0.1.3	0 % потерь	проверка маршрутов R2R3

Резервный маршрут и отказ линии

Для направления R1R2 был создан резервный путь через R3. В исходном оборудовании Huawei для выбора резервного маршрута используется параметр preference; в Linux аналогичный выбор среди статических маршрутов был воспроизведен с помощью metric — маршрут с меньшим значением выбирается первым.

```
# R1: заменить прежний маршрут парой с явными метриками
root@R1:/# ip route del 10.0.1.2/32 via 10.0.12.2
root@R1:/# ip route add 10.0.1.2/32 via 10.0.12.2 metric 60
root@R1:/# ip route add 10.0.1.2/32 via 10.0.13.3 metric 100

# R2: симметричная настройка обратного направления
root@R2:/# ip route del 10.0.1.1/32 via 10.0.12.1
root@R2:/# ip route add 10.0.1.1/32 via 10.0.12.1 metric 60
root@R2:/# ip route add 10.0.1.1/32 via 10.0.23.3 metric 100
```

После настройки в таблице одновременно видны основной и резервный маршруты. metric 60 и metric 100 различают записи с одинаковым префиксом; при прочих равных используется меньшее значение:

```
root@R1:/# ip route show 10.0.1.2/32
10.0.1.2 via 10.0.12.2 dev eth1 metric 60
10.0.1.2 via 10.0.13.3 dev eth2 metric 100

root@R1:/# ip route get 10.0.1.2 from 10.0.1.1
10.0.1.2 from 10.0.1.1 via 10.0.12.2 dev eth1 uid 0
    cache

root@R2:/# ip route show 10.0.1.1/32
10.0.1.1 via 10.0.12.1 dev eth1 metric 60
10.0.1.1 via 10.0.23.3 dev eth2 metric 100
```

Для моделирования отказа линия R1-R2 была административно отключена с обеих сторон:

```
ip link set eth1 down    # R1
ip link set eth1 down    # R2
```

После отключения интерфейс имеет административное состояние DOWN, а основной маршрут удалён из таблицы; остаётся маршрут с метрикой 100 через R3:

```
root@R1:/# ip -br link show eth1
eth1@if9          DOWN          be:57:9a:cd:7e:b8 <BROADCAST,MULTICAST>

root@R1:/# ip route show 10.0.1.2/32
10.0.1.2 via 10.0.13.3 dev eth2 metric 100

root@R1:/# ip route get 10.0.1.2 from 10.0.1.1
10.0.1.2 from 10.0.1.1 via 10.0.13.3 dev eth2 uid 0
    cache

root@R1:/# ping -c 3 -I 10.0.1.1 10.0.1.2
64 bytes from 10.0.1.2: icmp_seq=1 ttl=63 time=0.105 ms
64 bytes from 10.0.1.2: icmp_seq=2 ttl=63 time=0.081 ms
```

```
64 bytes from 10.0.1.2: icmp_seq=3 ttl=63 time=0.092 ms
3 packets transmitted, 3 received, 0% packet loss, time 2045ms
```

TTL ответа уменьшился с 64 до 63, поскольку на обратном пути появился промежуточный маршрутизатор R3. Наиболее наглядная диагностическая команда — `tracert`:

```
root@R1:~# traceroute -n -s 10.0.1.1 10.0.1.2
traceroute to 10.0.1.2 (10.0.1.2), 30 hops max, 60 byte packets
 1  10.0.13.3  0.026 ms  0.013 ms  0.009 ms
 2  10.0.1.2   0.040 ms  0.015 ms  0.012 ms
```

После отказа прямого сегмента трафик прошёл по пути R1 -> R3 -> R2.

При последующем `ip link set eth1 up` ядро восстановило подключенный маршрут 10.0.12.0/24, но не удалённый статический /32: его потребовалось добавить повторно. UP означает административное разрешение работы, а LOWER_UP — наличие работающего нижележащего канала; их различие подробно описано в справочном разделе.

Маршрут по умолчанию

После проверки резервирования линия R1–R2 была включена снова, а прямой обратный маршрут на R2 восстановлен. Специфические маршруты R1 к 10.0.1.2/32 были удалены. До добавления default route диагностическая команда подтвердила отсутствие подходящего пути:

```
root@R1:~# ip route get 10.0.1.2 from 10.0.1.1
RTNETLINK answers: Network is unreachable
```

Маршрут был добавлен отдельной командой, после чего сразу выведена вся таблица R1:

```
root@R1:~# ip route add default via 10.0.12.2

root@R1:~# ip route show
default via 10.0.12.2 dev eth1
10.0.1.3 via 10.0.13.3 dev eth2
10.0.12.0/24 dev eth1 proto kernel scope link src 10.0.12.1
10.0.13.0/24 dev eth2 proto kernel scope link src 10.0.13.1
```

При этом 10.0.1.2 достигается через маршрут по умолчанию, а для 10.0.1.3 Linux сохраняет более специфичный маршрут /32 через R3. Это демонстрирует правило longest prefix match: наиболее специфичный совпавший префикс имеет приоритет над 0.0.0.0/0.

После настройки default route тот же запрос стал разрешаться через R2:

```
root@R1:~# ip route get 10.0.1.2 from 10.0.1.1
10.0.1.2 from 10.0.1.1 via 10.0.12.2 dev eth1 uid 0
    cache

root@R1:~# ping -c 3 -I 10.0.1.1 10.0.1.2
64 bytes from 10.0.1.2: icmp_seq=1 ttl=64 time=0.132 ms
64 bytes from 10.0.1.2: icmp_seq=2 ttl=64 time=0.059 ms
64 bytes from 10.0.1.2: icmp_seq=3 ttl=64 time=0.070 ms
3 packets transmitted, 3 received, 0% packet loss, time 2031ms
```

Связь R1->R2 восстановилась; выбор default route подтверждён независимо командами `ip route get` и `ping`.

Итоговая конфигурация первой части

Адреса интерфейсов

Устройство	Интерфейс	IPv4-адрес
R1	lo	10.0.1.1/32

	eth1	10.0.12.1/24
	eth2	10.0.13.1/24
R2	lo	10.0.1.2/32
	eth1	10.0.12.2/24
	eth2	10.0.23.2/24
R3	lo	10.0.1.3/32
	eth1	10.0.13.3/24
	eth2	10.0.23.3/24

Полные таблицы маршрутизации уже приведены непосредственно после тех операций, которые их изменяли. Это позволяет сопоставить причину и результат и не смешивать несовместимые промежуточные конфигурации в одну «финальную» таблицу.

Контрольные вопросы первой части

Когда статический маршрут добавляется в таблицу IP-маршрутизации?

Статический маршрут может быть активным, когда указанный next hop достижим через рабочий интерфейс/существующий маршрут. Если следующий переход недоступен, такой маршрут не может использоваться для пересылки пакетов. В выполненном стенде это наблюдалось при отказе линии: после административного отключения интерфейса маршруты через него переставали участвовать в выборе пути, и использовался резервный маршрут.

Какой исходный IP будет выбран для ping без явного указания source?

Маршрутизатор выбирает исходный адрес в соответствии с выбранным выходным интерфейсом. Для прямого маршрута R1->R2 через сеть 10.0.12.0/24 источником стал бы адрес 10.0.12.1, а не loopback 10.0.1.1. Поэтому для проверки именно связи loopback-to-loopback источник указывался явно (ping -I 10.0.1.1 ...; в Huawei — параметр -a).

Вывод по первой части

В ходе первой части лабораторной работы была создана трехмаршрутизаторная IP-сеть в Docker-контейнерах без специализированного routing-daemon. Настроены физические и loopback-адреса, статические маршруты, резервный маршрут и маршрут по умолчанию. Экспериментально подтверждены необходимость обратного маршрута, автоматическое переключение на резервный путь при отказе линии и правило выбора наиболее специфичного маршрута. Использование Linux network namespace и veth позволило воспроизвести основные сетевые процессы исходной Huawei-лабораторной стандартными средствами ядра Linux.

Часть 2. Маршрутизация OSPF

Цель и план второй части

Цель второй части — настроить однозонную динамическую маршрутизацию OSPF на трех маршрутизаторах, проверить формирование соседств и автоматическое появление маршрутов, настроить аутентификацию OSPF, анонсирование маршрута по умолчанию и выбор пути на основании стоимости. Дополнительно проверяется автоматическая перестройка маршрутов после отказа и восстановления канала.

Топология и IPv4-адресация сохранены из первой части: 10.0.12.0/24 соединяет R1 и R2, 10.0.13.0/24 — R1 и R3, 10.0.23.0/24 — R2 и R3. Loopback-адреса: R1 — 10.0.1.1/32, R2 — 10.0.1.2/32, R3 — 10.0.1.3/32.

Подготовка FRRouting

Для реализации OSPF образ маршрутизатора был дополнен FRRouting. В работе использовалась версия FRR 8.4.4. Демон ospfd реализует OSPFv2, а zebra взаимодействует с таблицей маршрутизации ядра Linux и устанавливает рассчитанные маршруты в FIB. Для корректного запуска контейнеров использовались capabilities NET_ADMIN, NET_RAW и SYS_ADMIN.

```
FROM ubuntu:24.04

RUN apt-get update && DEBIAN_FRONTEND=noninteractive apt-get install -y \\\
    iproute2 iputils-ping traceroute procps tcpdump frr tini \\\
    && rm -rf /var/lib/apt/lists/*
```

В контейнерах были запущены только необходимые для данной работы демоны FRR. Проверка на каждом маршрутизаторе показала работающие процессы zebra и ospfd:

```
/usr/lib/frr/zebra -d -A 127.0.0.1 -f /etc/frr/zebra.conf
/usr/lib/frr/ospfd -d -A 127.0.0.1 -f /etc/frr/ospfd.conf
```

Настройка выполнялась параллельно в трех терминалах — по одному для R1, R2 и R3. Это позволило одновременно наблюдать изменение соседств и таблиц маршрутизации на разных устройствах.

Исходное состояние перед OSPF

После создания veth-соединений и назначения адресов непосредственно подключенные интерфейсы были доступны. Например, на R1 проверки двух соседей завершились без потерь:

```
root@R1:/# ip route show
10.0.12.0/24 dev eth1 proto kernel scope link src 10.0.12.1
10.0.13.0/24 dev eth2 proto kernel scope link src 10.0.13.1

root@R1:/# vtysh -c "show ip ospf neighbor"
% OSPF is not enabled in vrf default

root@R1:/# ip route get 10.0.1.2 from 10.0.1.1
RTNETLINK answers: Network is unreachable
```

Перед второй частью были удалены все статические записи: физическая связность сохранилась, процесс OSPF ещё не был включён, а маршрута к loopback R2 не было. FRR-демоны были перезапущены с пустой RIB, чтобы записи первой части не влияли на опыт.

Настройка базового OSPF

На всех трех устройствах был создан процесс OSPF и явно задан Router ID, совпадающий с loopback-адресом. Все сети были помещены в backbone-area 0.0.0.0.

```
R1(config)# router ospf
R1(config-router)# ospf router-id 10.0.1.1
R1(config-router)# network 10.0.1.1/32 area 0.0.0.0
R1(config-router)# network 10.0.12.0/24 area 0.0.0.0
R1(config-router)# network 10.0.13.0/24 area 0.0.0.0
```

```

R2(config)# router ospf
R2(config-router)# ospf router-id 10.0.1.2
R2(config-router)# network 10.0.1.2/32 area 0.0.0.0
R2(config-router)# network 10.0.12.0/24 area 0.0.0.0
R2(config-router)# network 10.0.23.0/24 area 0.0.0.0

R3(config)# router ospf
R3(config-router)# ospf router-id 10.0.1.3
R3(config-router)# network 10.0.1.3/32 area 0.0.0.0
R3(config-router)# network 10.0.13.0/24 area 0.0.0.0
R3(config-router)# network 10.0.23.0/24 area 0.0.0.0

```

router-id — уникальный 32-битный идентификатор OSPF-маршрутизатора; здесь он намеренно совпадает с устойчивым loopback-адресом. network ... area не создаёт сеть, а выбирает интерфейсы, на которых запускается OSPF, и относит их к backbone-area 0.0.0.0.

После запуска на broadcast-интерфейсах кратковременно наблюдались состояния Init и 2-Way: маршрутизаторы обменивались Hello и выбирали DR/BDR. После сходимости на R1 сформировались два соседства Full:

```

root@R1:/# vtysh -c "show ip ospf neighbor"

```

Neighbor ID	Pri	State	Dead Time	Address	Interface
10.0.1.2	1	Full/DR	35.377s	10.0.12.2	eth1:10.0.12.1
10.0.1.3	1	Full/DR	35.508s	10.0.13.3	eth2:10.0.13.1

Full означает синхронизированную LSDB; DR — роль соседа Designated Router на данном broadcast-сегменте. Dead Time показывает оставшееся время, после которого сосед будет признан недоступным без новых Hello.

FRR показывает свою RIB командой show ip route ospf, а ядро — командой ip route show proto ospf:

```

root@R1:/# vtysh -c "show ip route ospf"
0>* 10.0.1.2/32 [110/10] via 10.0.12.2, eth1, weight 1
0>* 10.0.1.3/32 [110/10] via 10.0.13.3, eth2, weight 1
0>* 10.0.23.0/24 [110/20] via 10.0.12.2, eth1, weight 1
    *                      via 10.0.13.3, eth2, weight 1

root@R1:/# ip route show proto ospf
10.0.1.2 nhid 38 via 10.0.12.2 dev eth1 metric 20
10.0.1.3 nhid 34 via 10.0.13.3 dev eth2 metric 20
10.0.23.0/24 nhid 39 metric 20
    nexthop via 10.0.13.3 dev eth2 weight 1
    nexthop via 10.0.12.2 dev eth1 weight 1

```

В FRR 0 обозначает OSPF, > — выбранный маршрут, * — установленный в FIB; [110/10] содержит administrative distance и OSPF cost. Для 10.0.23.0/24 установлены два равноценных next hop с weight 1, то есть ECMP.

После появления маршрута проверка loopback стала успешной:

```

root@R1:/# ping -c 3 -I 10.0.1.1 10.0.1.2
64 bytes from 10.0.1.2: icmp_seq=1 ttl=64 time=0.083 ms
64 bytes from 10.0.1.2: icmp_seq=2 ttl=64 time=0.067 ms
64 bytes from 10.0.1.2: icmp_seq=3 ttl=64 time=0.070 ms
3 packets transmitted, 3 received, 0% packet loss, time 2035ms

```

В отличие от первой части, к удалённым loopback не выполнялось ни одной команды ip route add: маршруты рассчитал ospfd, а zebra установила их в FIB ядра.

Аутентификация OSPF

Аутентификация настраивалась намеренно поэтапно. Сначала режим message-digest и ключ были включены только на двух интерфейсах R1:

```
R1(config)# interface eth1
R1(config-if)# ip ospf authentication message-digest
R1(config-if)# ip ospf message-digest-key 1 md5 HClA-Datacom

R1(config)# interface eth2
R1(config-if)# ip ospf authentication message-digest
R1(config-if)# ip ospf message-digest-key 1 md5 HClA-Datacom
```

message-digest-key 1 задаёт идентификатор ключа 1; md5 — алгоритм, а HClA-Datacom — общий секрет. На обоих концах линии должны совпадать режим, key ID и секрет. Команда проверки показала, что режим применён, но до истечения Dead Timer старые соседства ещё могли временно отображаться:

```
root@R1:/# vtysh -c "show ip ospf interface eth1"
eth1 is up
  Network Type BROADCAST, Cost: 10
  Timer intervals configured, Hello 10s, Dead 40s
  Cryptographic authentication enabled
  Algorithm:MD5

root@R1:/# vtysh -c "show ip ospf neighbor"
Neighbor ID  Pri State  Dead Time Address Interface

root@R1:/# ip route show proto ospf
```

Пустые результаты после 40 секунд означают, что R2 и R3 продолжали отправлять неаутентифицированные Hello: R1 их отвергал, соседства истекли, а изученные OSPF-маршруты были удалены из FIB.

Затем тот же интерфейсный режим и ключ были настроены на R2. Связь R1–R2 восстановилась, а R2–R3 осталась разорванной:

```
root@R2:/# vtysh -c "show ip ospf neighbor"

Neighbor ID  Pri State          Dead Time Address      Interface
10.0.1.1     1 Full/Backup    30.424s 10.0.12.1    eth1:10.0.12.2
```

На R3, как и в методичке Huawei, применена аутентификация всей area 0. В FRR режим задаётся на уровне area, но сами ключи всё равно назначаются интерфейсам:

```
R3(config-router)# area 0.0.0.0 authentication message-digest
R3(config)# interface eth1
R3(config-if)# ip ospf message-digest-key 1 md5 HClA-Datacom
R3(config)# interface eth2
R3(config-if)# ip ospf message-digest-key 1 md5 HClA-Datacom

root@R3:/# vtysh -c "show ip ospf neighbor"
Neighbor ID  Pri State          Dead Time Address      Interface
10.0.1.1     1 Full/Backup    35.498s 10.0.13.1    eth1:10.0.13.3
10.0.1.2     1 Full/Backup    35.609s 10.0.23.2    eth2:10.0.23.3
```

Последний вывод подтверждает восстановление обоих соседств после согласования параметров аутентификации.

Анонсирование маршрута по умолчанию

R1 использовался как граничный маршрутизатор и был настроен на безусловное анонсирование маршрута 0.0.0.0/0 в OSPF:

```
R1(config)# router ospf
R1(config-router)# default-information originate always
```

Параметр `always` позволяет распространять default route даже при отсутствии собственного маршрута по умолчанию в основной таблице R1. Это является функциональным аналогом Huawei-команды `default-route-advertise always`.

Сначала проверено, что в Linux-таблице самого R1 default route действительно отсутствует. Тем не менее R2 и R3 получили внешний OSPF-маршрут:

```
root@R1:/# ip route show default

root@R2:/# vtysh -c "show ip route 0.0.0.0/0"
Routing entry for 0.0.0.0/0
  Known via "ospf", distance 110, metric 1, best
  * 10.0.12.1, via eth1, weight 1

root@R2:/# ip route show default
default via 10.0.12.1 dev eth1 proto ospf metric 20
```

На R3 результат аналогичен:

```
root@R3:/# vtysh -c "show ip route 0.0.0.0/0"
Routing entry for 0.0.0.0/0
  Known via "ospf", distance 110, metric 1, best
  * 10.0.13.1, via eth1, weight 1

root@R3:/# ip route show default
default via 10.0.13.1 dev eth1 proto ospf metric 20
```

Полученная FRR OSPF-информация была рассчитана `routing-daemon` и затем установлена `zebra` в таблицу маршрутизации Linux.

Управление выбором маршрута с помощью OSPF cost

В FRR исходная стоимость каждого физического OSPF-интерфейса составляла 10. До изменения R1 выбирал прямой путь стоимостью 10:

```
root@R1:/# vtysh -c "show ip route 10.0.1.2/32"
Routing entry for 10.0.1.2/32
  Known via "ospf", distance 110, metric 10, best
  * 10.0.12.2, via eth1, weight 1

root@R1:/# traceroute -n -s 10.0.1.1 10.0.1.2
traceroute to 10.0.1.2 (10.0.1.2), 30 hops max, 60 byte packets
 1  10.0.1.2  0.563 ms  0.036 ms  0.035 ms
```

Альтернатива R1->R3->R2 имеет стоимость $10 + 10 = 20$. Поэтому для переключения стоимость прямого интерфейса должна быть больше 20; было выбрано 30:

```
R1(config)# interface eth1
R1(config-if)# ip ospf cost 30
```

После пересчёта SPF маршрут изменился:

```
root@R1:/# vtysh -c "show ip route 10.0.1.2/32"
Routing entry for 10.0.1.2/32
  Known via "ospf", distance 110, metric 20, best
  * 10.0.13.3, via eth2, weight 1

root@R1:/# ip route get 10.0.1.2 from 10.0.1.1
```

```
10.0.1.2 from 10.0.1.1 via 10.0.13.3 dev eth2 uid 0
cache
```

Суммарная стоимость пути через R3 равна $10 + 10 = 20$, что меньше стоимости прямой линии 30. Трассировка подтвердила изменение пути:

```
root@R1:/# traceroute -n -s 10.0.1.1 10.0.1.2
traceroute to 10.0.1.2 (10.0.1.2), 30 hops max, 60 byte packets
 1  10.0.13.3  0.561 ms  0.028 ms  0.018 ms
 2  10.0.1.2   0.027 ms  0.012 ms  *
```

В Linux/FRR конечный R2 ответил на второй hop своим loopback-адресом 10.0.1.2; при этом фактический путь соответствует R1 -> R3 -> R2.

Асимметричный обратный маршрут

Изменение стоимости выполнялось только на интерфейсе eth1 маршрутизатора R1. Поэтому стоимость направления R2->R1 не изменилась. Проверка на R2 показала:

```
root@R2:/# ip route get 10.0.1.1 from 10.0.1.2
10.0.1.1 from 10.0.1.2 via 10.0.12.1 dev eth1 uid 0
cache
```

В результате прямой и обратный пути стали различаться:

```
ICMP request: R1 -> R3 -> R2
ICMP reply:   R2 -> R1
```

Это демонстрирует возможность асимметричной маршрутизации: стоимость OSPF задается для исходящего интерфейса конкретного маршрутизатора и не обязана быть одинаковой в обратном направлении.

Проверка отказоустойчивости OSPF

Для моделирования отказа линия R1-R3 была административно отключена с двух концов:

```
root@R1:/# ip link set eth2 down
root@R3:/# ip link set eth1 down
```

После отказа сосед R3 исчез из списка, а соседство R1-R2 осталось в состоянии Full. OSPF автоматически заменил недоступный путь через R3 прямым путем R1->R2, несмотря на его более высокую стоимость:

```
root@R1:/# ip -br link show eth2
eth2@if11      DOWN  8a:88:09:06:af:62 <BROADCAST,MULTICAST>

root@R1:/# vtysh -c "show ip ospf neighbor"
Neighbor ID Pri State   Dead Time Address        Interface
10.0.1.2     1 Full/DR   37.984s 10.0.12.2     eth1:10.0.12.1

root@R1:/# vtysh -c "show ip route 10.0.1.2/32"
Routing entry for 10.0.1.2/32
  Known via "ospf", distance 110, metric 30, best
  * 10.0.12.2, via eth1, weight 1

root@R1:/# ping -c 3 -I 10.0.1.1 10.0.1.2
64 bytes from 10.0.1.2: icmp_seq=1 ttl=64 time=0.053 ms
64 bytes from 10.0.1.2: icmp_seq=2 ttl=64 time=0.083 ms
64 bytes from 10.0.1.2: icmp_seq=3 ttl=64 time=0.062 ms
3 packets transmitted, 3 received, 0% packet loss, time 2079ms
```

После восстановления интерфейса:

```
root@R1:/# ip link set eth2 up
root@R3:/# ip link set eth1 up
```

OSPF повторно сформировал соседство с R3. После завершения сходимости оба соседа R1 находились в состоянии Full, а более дешевый маршрут через R3 был возвращен автоматически:

```
root@R1:/# vtysh -c "show ip ospf neighbor"
Neighbor ID  Pri State   Dead Time Address    Interface
10.0.1.2      1 Full/DR   31.734s 10.0.12.2  eth1:10.0.12.1
10.0.1.3      1 Full/DR   33.868s 10.0.13.3  eth2:10.0.13.1

root@R1:/# vtysh -c "show ip route 10.0.1.2/32"
Routing entry for 10.0.1.2/32
  Known via "ospf", distance 110, metric 20, best
  * 10.0.13.3, via eth2, weight 1

root@R1:/# ip route get 10.0.1.2 from 10.0.1.1
10.0.1.2 from 10.0.1.1 via 10.0.13.3 dev eth2 uid 0
cache
```

Эксперимент показывает основное преимущество динамической маршрутизации по сравнению со статической: после изменения топологии OSPF самостоятельно пересчитывает SPF и изменяет активный маршрут без ручного добавления нового `ip route`.

Ответ на контрольный вопрос второй части

На шаге изменения стоимости маршрутизатор R2 использует для возврата ICMP-пакетов к loopback R1 прямой маршрут через 10.0.12.1 (eth1). Это подтверждено командой `ip route get 10.0.1.1 from 10.0.1.2`.

Причина заключается в том, что стоимость была увеличена только на исходящем интерфейсе R1->R2. Для R1 прямой путь к R2 стал иметь стоимость 30, поэтому был выбран путь R1->R3->R2 со стоимостью 20. На R2 стоимость собственного прямого интерфейса в сторону R1 не менялась, поэтому обратный ICMP reply передается непосредственно R2->R1. В эксперименте сформировалась асимметричная маршрутизация.

Вывод по второй части

Во второй части на том же трехмаршрутизаторном Docker-стенде была реализована однозонная маршрутизация OSPF с помощью FRRouting. После запуска `zebra` и `ospfd` маршрутизаторы автоматически обменялись информацией о сетях и установили маршруты к удаленным loopback-интерфейсам. Была проверена MD5-аутентификация OSPF, распространение маршрута по умолчанию с R1, ECMP для равноценных путей и изменение выбора маршрута через настройку OSPF cost.

Изменение стоимости показало, что OSPF выбирает путь по суммарной метрике, а прямой и обратный маршруты могут различаться. При отключении предпочтительного канала OSPF автоматически переключил трафик на оставшийся путь без потери связности, а после восстановления интерфейса вновь выбрал маршрут с меньшей стоимостью. В отличие от статической маршрутизации первой части, динамический протокол самостоятельно реагирует на изменение топологии и поддерживает актуальные маршруты в таблице Linux.

Справочная информация по командам и выводу

Параметры виртуального стенда

`ip link add r1-r2 type veth peer name r2-r1` создаёт пару виртуальных Ethernet-интерфейсов: пакет, отправленный в один конец, появляется на другом. `ip link set ... netns PID` переносит конец пары в `network namespace` процесса с указанным PID; после этого интерфейс виден только соответствующему контейнеру.

Контейнерный параметр `--network none` запрещает Docker создавать собственный `eth0` и маршруты, чтобы топология полностью определялась вручную. Capability `NET_ADMIN` разрешает изменение адресов, интерфейсов и маршрутов, `NET_RAW` — raw-сокеты для ICMP и OSPF, `SYS_ADMIN` потребовалась используемой реализации FRR. `Sysctl net.ipv4.ip_forward=1` включает пересылку IPv4 между интерфейсами. В Dockerfile ключ `apt-get -y` автоматически подтверждает установку перечисленных пакетов, а `rm -rf /var/lib/apt/lists/*` уменьшает размер образа, удаляя скачанные индексы пакетов после установки.

Как Linux создаёт маршруты при назначении адреса

Команда `ip addr add 10.0.12.1/24 dev eth1` передаёт ядру запрос Netlink на добавление адреса. Если интерфейс существует и префикс не помечен `perfixroute`, ядро в рамках той же операции создаёт связанные записи. Это происходит не при загрузке ОС как отдельный поздний этап, а непосредственно при назначении адреса — из загрузочного скрипта, сетевого менеджера или команды администратора.

Для адреса `10.0.12.1/24` появляются:

- `10.0.12.0/24` в таблице `main` — достижимость всей непосредственно подключенной сети;
- `local 10.0.12.1/32` в таблице `local` — доставка пакетов самому Linux-хосту;
- `broadcast 10.0.12.255/32` в таблице `local` — обработка directed broadcast данного префикса.

Таблица `local` имеет более высокий приоритет в стандартных правилах RPDБ и обычно просматривается отдельно командой `ip route show table local`; поэтому одна команда `ip route show` не показывает все три записи.

Расшифровка строки `10.0.12.0/24 dev eth1 proto kernel scope link src 10.0.12.1`:

Поле	Значение
<code>10.0.12.0/24</code>	префикс назначения; /24 означает 24 единичных бита маски (255.255.255.0)
<code>dev eth1</code>	выходной интерфейс
<code>proto kernel</code>	источник записи — ядро, а не команда статического маршрута или <code>routing-daemon</code>
<code>scope link</code>	назначение находится непосредственно на данном канале; промежуточный шлюз не нужен
<code>src 10.0.12.1</code>	предпочтительный локальный адрес источника, если приложение не выбрало другой
<code>scope host</code>	маршрут действителен только внутри локального хоста; типичен для собственных адресов
<code>local / broadcast</code>	тип маршрута: доставка локальному стеку или широковещательная доставка

При административном `ip link set eth1 down` адрес остаётся назначенным, но связанные активные маршруты и вручную созданные маршруты через этот интерфейс могут быть удалены из FIB. После `up` ядро восстанавливает собственный `connected route`, но удалённый статический маршрут необходимо создать снова.

Параметры команд Linux

Запись	Назначение
<code>ip -br addr</code>	<code>-br (brief)</code> включает компактный однострочный формат интерфейсов
<code>ip route add</code>	добавляет запись; <code>via</code> задаёт next hop, <code>dev</code> — интерфейс, /32 — ровно один IPv4-адрес
<code>ip route get ... from ...</code>	показывает фактический выбор ядра для пакета с указанными назначением и источником, но пакет не отправляет
<code>uid 0</code>	поиск выполнен от имени UID 0; policy routing может учитывать UID

cache	выведен результат разрешённого route lookup; это не отдельная старая глобальная таблица route cache
metric 60	предпочтение среди Linux-маршрутов одного префикса; меньшее значение предпочтительнее
ip link set ... down/up	административно выключает/включает интерфейс
UP	интерфейс административно включён
LOWER_UP	нижний уровень сообщает о работающем канале; UP без LOWER_UP не гарантирует передачу
ping -c 3	-c (count) завершает программу после трёх echo request
ping -I 10.0.1.1	-I выбирает исходный адрес/интерфейс; использован для проверки именно loopback-to-loopback
icmp_seq	порядковый номер ICMP echo
ttl	Time To Live ответа; каждый маршрутизатор уменьшает TTL на единицу
rtt min/avg/max/mdev	минимальное, среднее, максимальное время round trip и mean deviation
tracert -n	-n запрещает обратные DNS-запросы и оставляет числовые адреса
tracert -s ADDRESS	-s задаёт адрес источника пробных пакетов
* в tracert	на конкретную пробу не получен ответ за время ожидания; это не обязательно означает потерю пользовательского трафика

Флаг -W, встречавшийся в предыдущей версии отчёта, исключён: он задавал тайм-аут ожидания отдельного ответа и не был нужен для смысла опыта. Оставлены только -c, необходимый для конечного автоматизированного запуска, и -I, необходимый для выбора loopback-источника.

FRRouting, OSPF и его диагностические поля

ospfd строит соседства, хранит LSDB и рассчитывает SPF. zebra объединяет маршруты протоколов в RIB FRR и через Netlink программирует FIB ядра. vtysh предоставляет общий CLI. В командах запуска демонов -d переводит процесс в фоновый режим, -A 127.0.0.1 ограничивает адрес VTY, -f FILE выбирает файл конфигурации. В vtysh -c "..." параметр -c означает выполнение одной CLI-команды и не связан с ping -c.

Поле	Смысл
0	маршрут известен через OSPF
>	FRR выбрал маршрут как лучший в своей RIB
*	маршрут передан в FIB
[110/10]	administrative distance 110 и OSPF cost 10
proto ospf	в таблицу ядра запись установлена процессом маршрутизации OSPF через FRR
metric 20 y Linux	служебная метрика Netlink, с которой FRR установила маршрут; OSPF cost следует смотреть в RIB FRR
nhid	идентификатор kernel nexthop object, используемого маршрутом
nexthop via	один из шлюзов составного ECMP-маршрута
weight 1	равный вес next hop; при двух записях обе участвуют в ECMP
Pri	OSPF interface priority при выборе DR/BDR
Init	Hello соседа получен, но собственный Router ID ещё не увиден в его Hello
2-Way	двусторонний Hello-обмен подтверждён
Full	adjacency сформирована и LSDB синхронизирована
DR / Backup	Designated Router и Backup Designated Router на broadcast-сегменте
Dead Time	оставшееся время до признания соседа недоступным без Hello
RXmtL/RqstL/DBsmL	длины списков retransmission, link-state request и database summary; нули характерны для устойчивого соседства

`default-information originate always` создаёт и распространяет OSPF default route даже без `0.0.0.0/0` в таблице R1. Это не создаёт у R1 реального выхода в Интернет: команда только сообщает соседям, что неизвестные назначения следует отправлять R1.

OSPF cost является направленной стоимостью исходящего интерфейса. Поэтому изменение cost только на R1 изменило путь запроса R1->R2, но не обязало R2 использовать тот же путь для ответа. После отказа или восстановления линии Full и маршруты появляются не мгновенно: нужны Hello-обмен, выборы DR/BDR, синхронизация LSDB и новый расчёт SPF.